CYB 304: Information and Big Data Security

Laboratory Assessment Report

Name: Ita Arthur

Matriculation number: 2024/B/CYB/0373

Student ID: 30160052

Course: CYB 304 Big Data and Information Security

Date: 8 November 2026

1. Introduction

This report covers my work on the CYB 304 laboratory assessment, which walks through the Big Data lifecycle from raw data acquisition and discovery to distributed analytics, machine learning, a NoSQL-backed web application and, finally, securing the platform. I completed the lab on my own and used a single dataset for every task so that the results of one task could feed into the next.

The dataset is the **Telco Customer Churn** dataset from Kaggle, which has 7,043 customer records and 21 attributes. My working question for the whole lab was: which customers are most likely to leave (churn), and what factors drive that behaviour? Churn data was a good fit for a security course because it contains customer identifiers and billing details, so it also raised realistic privacy and access-control issues.

Lab environment

Component	Details
Operating system	Kali Linux running as a VirtualBox guest
Resources allocated	4 GB RAM, 2 vCPUs, 80 GB virtual disk
Software stack	Java 21, Hadoop 3.3.6 (HDFS and YARN, single-node), Spark 3.5.9 (PySpark), R with Shiny, MongoDB 4.4 in Docker, Python Flask

I ran Hadoop in pseudo-distributed mode, meaning all the daemons (NameNode, DataNode, ResourceManager and NodeManager) ran on the same VM. This is obviously not a real cluster, but it behaves the same way from the user's point of view, so the commands and concepts carry over to a multi-node setup.

Environment setup

Before starting the tasks I checked the resources available to the VM. `free -h` reported 3.8 GiB of RAM and 4.9 GiB of swap, `nproc` confirmed 2 CPU cores, and `df -h /` showed a 79 GB root filesystem with 51 GB free.

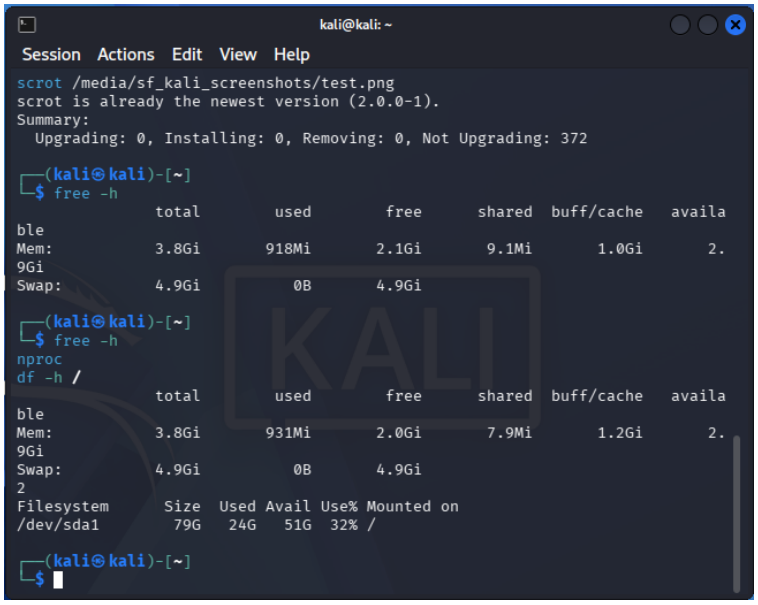


Figure 1: Checking VM resources: memory and swap, CPU cores and disk space

Hadoop needs a Java runtime, so I installed OpenJDK together with SSH and pdsh. The installation set Java 25 as the default, which is too new for Hadoop 3.3.6, so I used `update-alternatives` to switch the system default to Java 21 and confirmed it with `java -version` and `javac -version`.

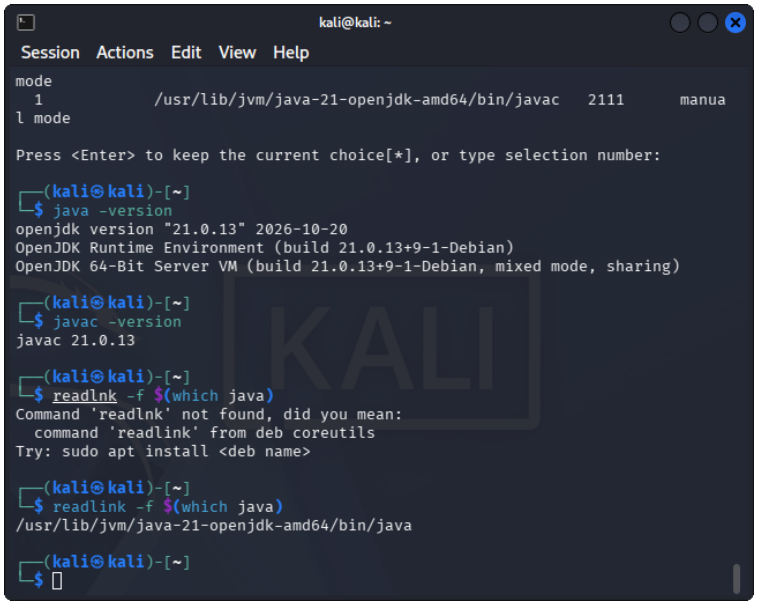


Figure 2: Java 21.0.13 confirmed as the active runtime and compiler

Hadoop's start-up scripts use SSH to launch the daemons, even on a single node, so I generated an RSA key pair, added it to authorized_keys and tested a passwordless login to localhost.

I then downloaded Hadoop 3.3.6 (696 MB), extracted it to /opt/hadoop and changed its ownership to my user. I set JAVA_HOME, HADOOP_HOME, HADOOP_CONF_DIR and SPARK_HOME in ~/.zshrc and added the Hadoop and Spark binaries to the PATH. Finally, I configured hadoop-env.sh with the Java 21 path (plus an --add-opens option needed on newer Java versions), set `fs.defaultFS` to hdfs://localhost:9000 in core-site.xml, and limited YARN to 2048 MB of memory to suit the 4 GB VM.

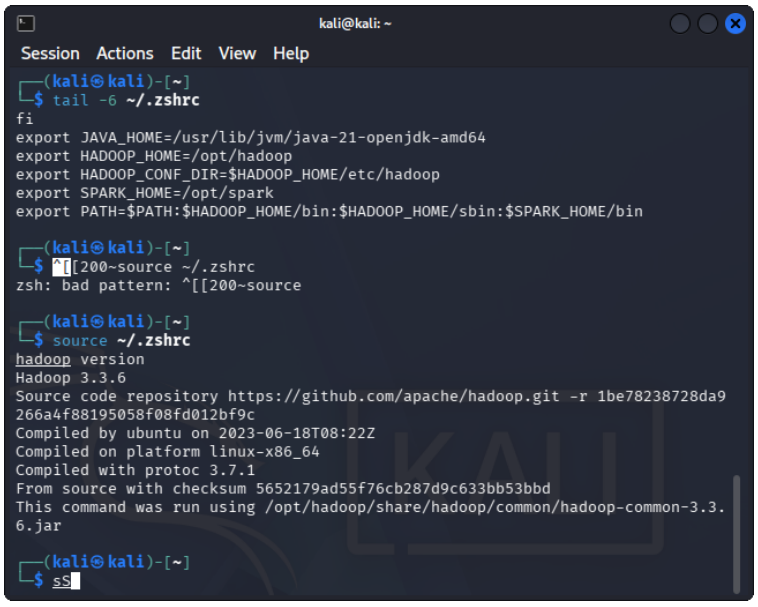


Figure 3: Environment variables in ~/.zshrc and hadoop version confirming Hadoop 3.3.6

In hdfs-site.xml I set the replication factor to 1, since a single node cannot hold more than one copy of each block, and pointed the NameNode and DataNode storage directories to ~/hadoopdata. In mapred-site.xml I set YARN as the MapReduce framework.

After formatting the NameNode, I started HDFS and YARN with `start-dfs.sh` and `start-yarn.sh`. `jps` confirmed that all five daemons were running: NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager. The NameNode web interface on port 9870 also showed the filesystem as active. It is worth noting that the same page reports “Security is off”, which is the default and one of the issues I come back to in Task 7.

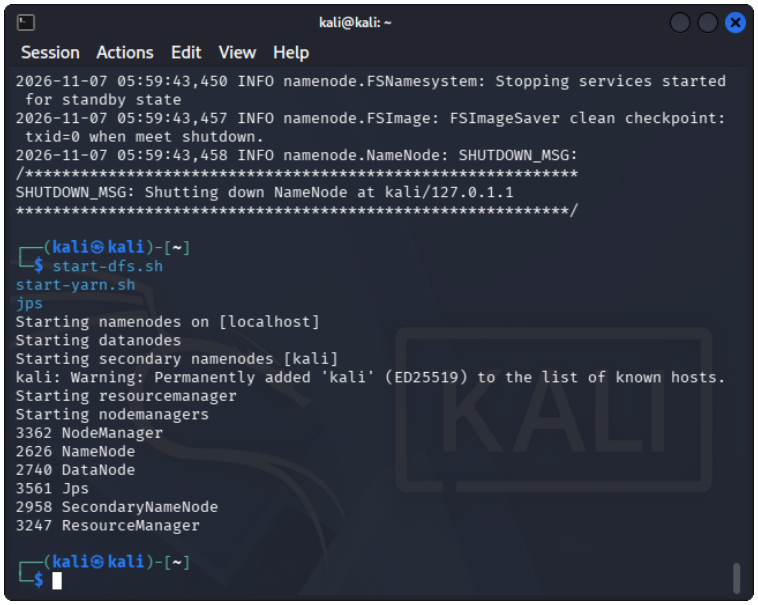


Figure 4: Starting HDFS and YARN; jps lists all five Hadoop daemons

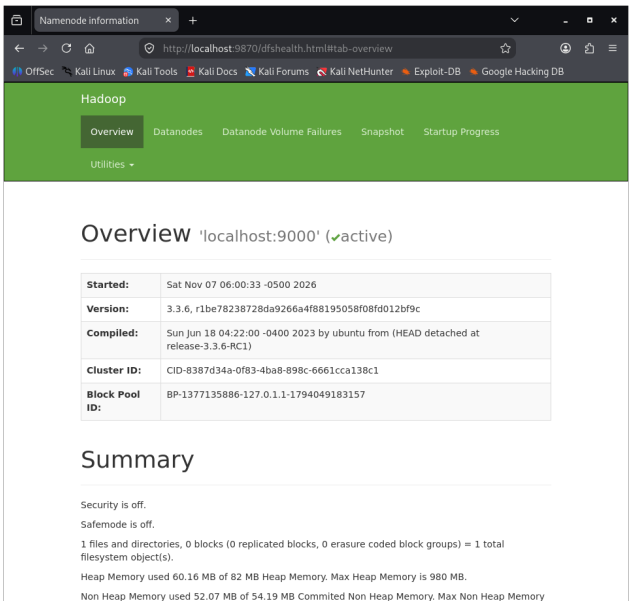


Figure 5: NameNode web UI at localhost:9870: Hadoop 3.3.6 active, with security off by default

Lastly, I extracted Spark 3.5.9 to /opt/spark and confirmed the installation with spark-submit --version.

2. Task 1: Data acquisition and ingestion

Procedure

1 Downloaded the Telco churn CSV from Kaggle and copied it into the VM.

2 Profiled the raw file from the command line using `head`, `wc -l`, `file` and `awk` before loading it anywhere.

3 Created a second data source in JSON format (the first 1,000 records) so that I was ingesting more than one file type, as the lab requires.

4 Ingested both files into HDFS with `hdfs dfs -put` and verified the upload with `hdfs dfs -ls` and `hdfs dfs -du -h`.

Observations

- The CSV contains 7,044 lines, which is 7,043 records plus a header row, across 21 columns.
- The target variable (Churn) is imbalanced: 5,174 “No” against 1,869 “Yes”, roughly a 73/27 split. I noted this early because it affects how the model in Task 5 should be evaluated.
- `awk` showed 11 records with an empty `TotalCharges` field. Because of these blanks, a column that should be numeric gets treated as text.
- Both files landed in HDFS under /user/kali/raw, at 954.6 K for the CSV and 512.6 K for the JSON.

Challenges of handling raw data. The main lesson from this task is that raw data cannot be trusted at face value. Missing values were not marked as null but hidden as empty strings, there is no schema enforcing data types, the classes are unbalanced, and the same data in CSV and JSON needs different parsing logic. Doing this profiling step before ingestion meant I already knew what to look for in the discovery phase.

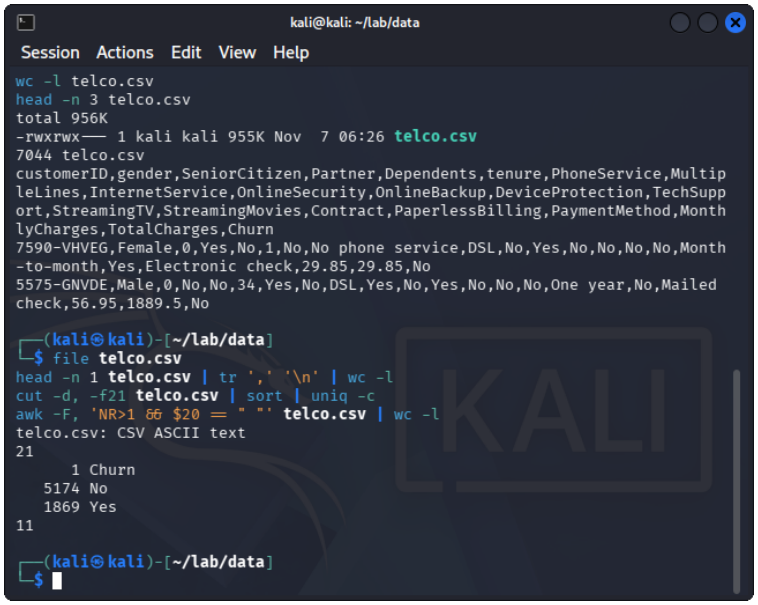


Figure 6: Profiling with file, tr, cut and awk: CSV ASCII text, 21 columns, churn counts 5,174 No / 1,869 Yes, and 11 blank TotalCharges values

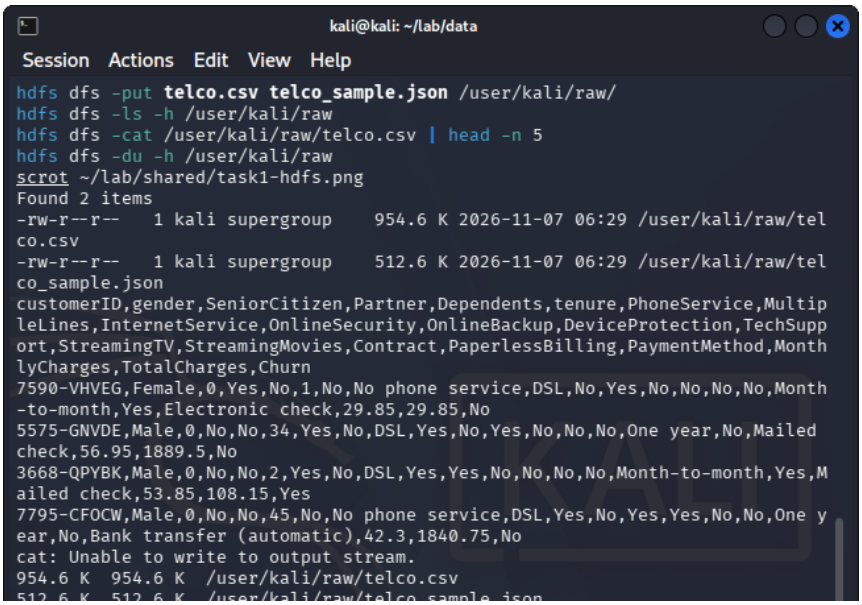


Figure 7: Ingesting both files into HDFS, then listing, sampling the first rows and checking sizes with du

3. Task 2: Initiating data discovery

Procedure. I used PySpark to read the CSV directly from HDFS (hdfs://localhost:9000) with schema inference turned on. From there I inspected the inferred schema, generated summary statistics, sampled rows, checked for nulls and blanks, and grouped churn by the main categorical attributes.

Observations

- Spark loaded all 7,043 records and 21 columns. Schema inference typed TotalCharges as a string, which confirmed the data quality issue I found in Task 1.
- All 11 blank TotalCharges records have `tenure = 0` and `Churn = No`. These are not errors but brand-new customers who had not received a bill yet, so the right fix is to treat their total as 0 rather than drop them.
- A standard null check returned zero nulls in every column. This was an important finding: if I had relied on the null check alone, the missing values would have gone unnoticed, because they were stored as blank strings.
- Mean tenure is about 32 months and the mean monthly charge is about $64.76.

Contract type	Churn rate
Month-to-month	42.7%
One year	11.3%
Two year	2.8%

Internet service	Churn rate
Fiber optic	about 42%
DSL	about 19%
None	about 7%

Early-stage insights. Churn is heavily concentrated among month-to-month customers and fiber optic subscribers. Customers on longer contracts rarely leave. At this point these are only correlations, but they gave me a hypothesis to test in the analytics and machine learning tasks.

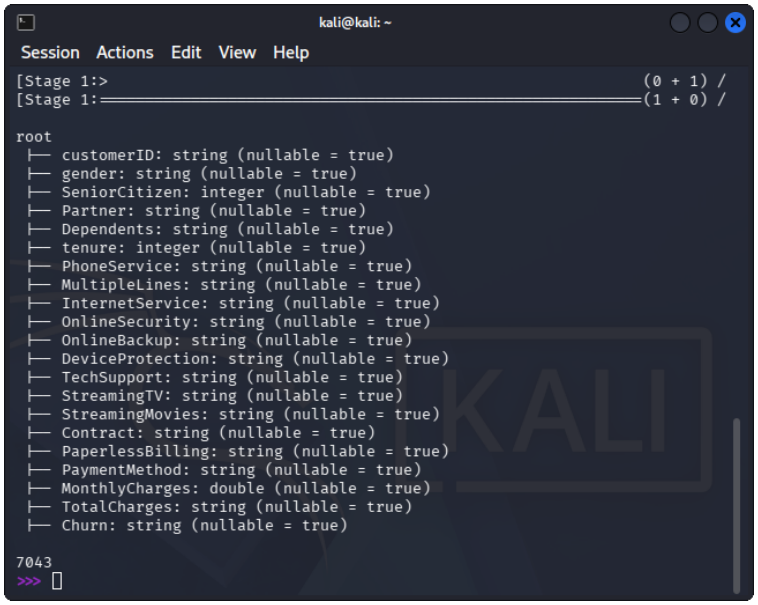


Figure 8: printSchema output: TotalCharges inferred as string; count returns 7,043 rows

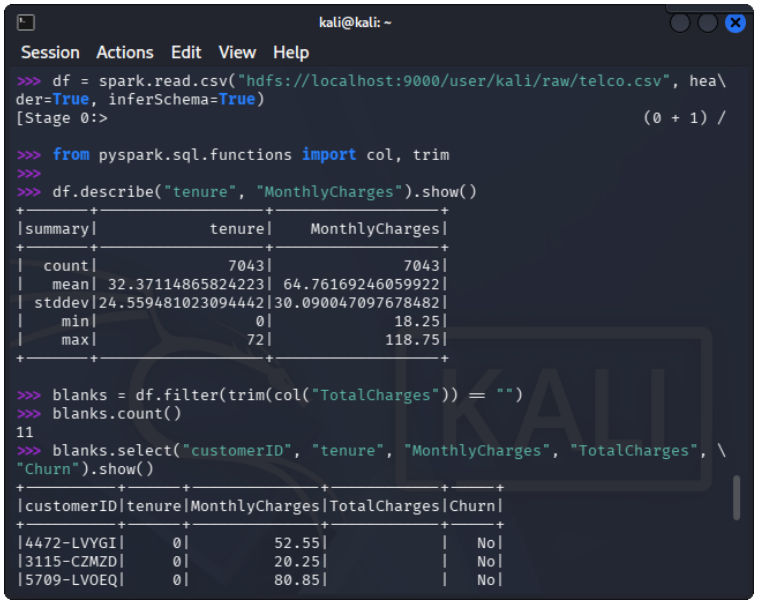


Figure 9: Summary statistics for tenure and MonthlyCharges, followed by the blank TotalCharges check

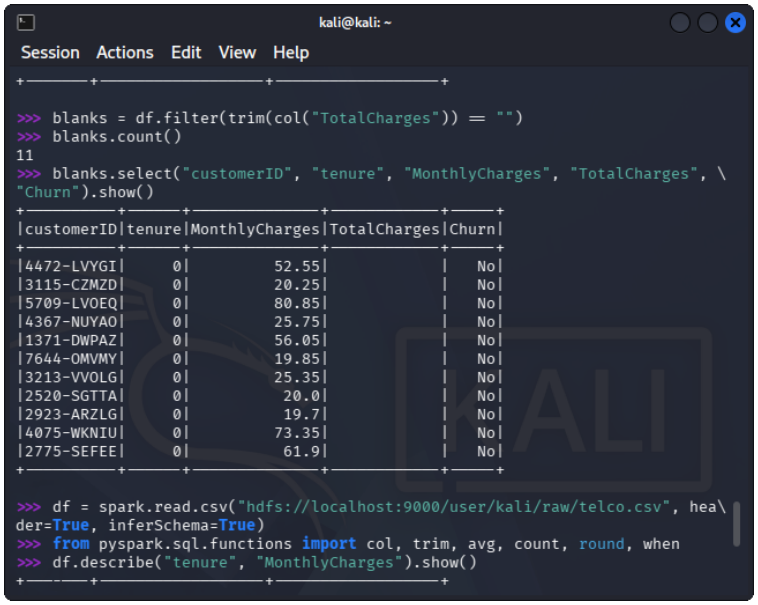


Figure 10: All 11 blank TotalCharges records have tenure 0 and Churn = No

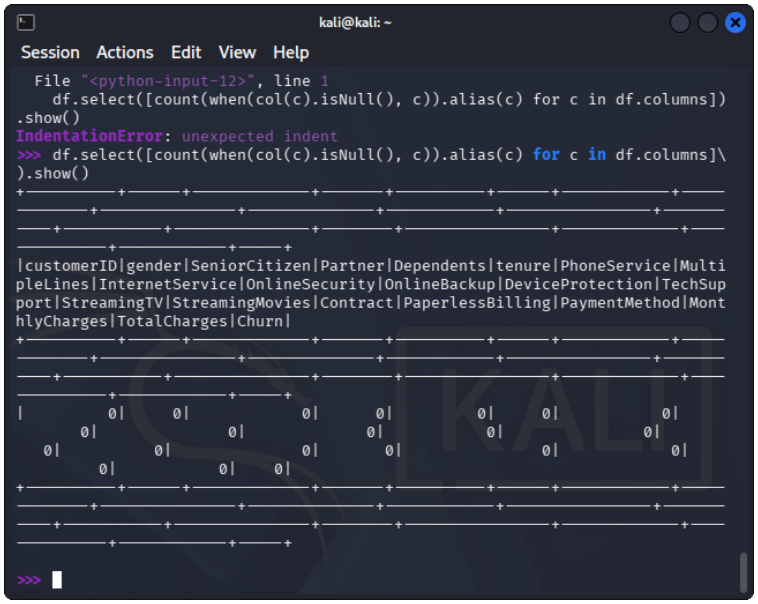


Figure 11: Null check across every column returns 0, even though 11 values are blank

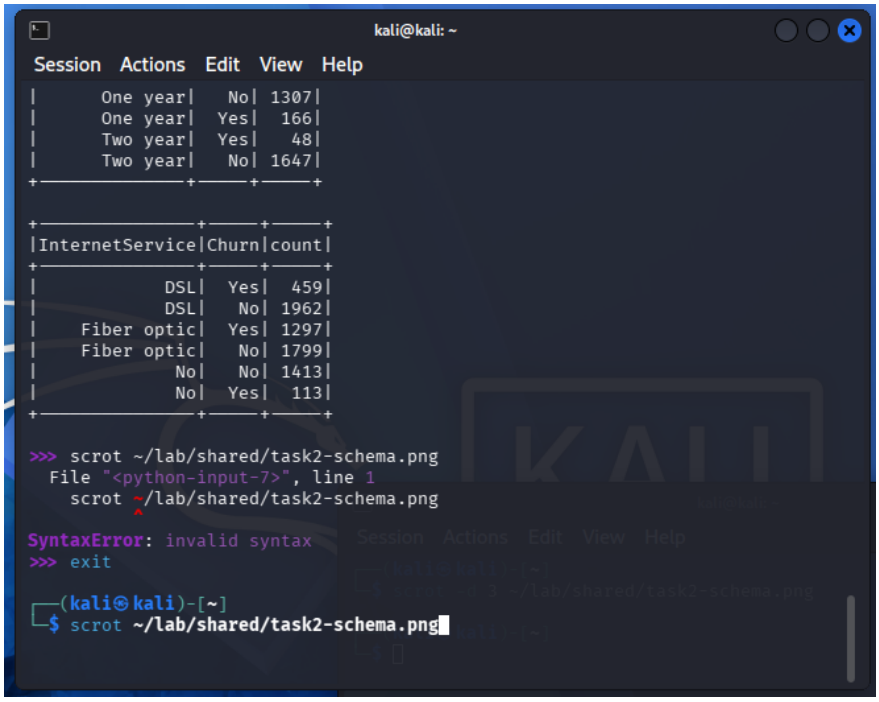


Figure 12: Churn counts broken down by contract type and internet service

4. Task 3: Big Data analytics with Hadoop and Spark

Procedure. I first cleaned the data by casting TotalCharges to a double, then applied Spark transformations (`filter` , `groupBy` , `agg`) and triggered them with actions (`count` , `show` , `collect`).

Results

- 3,875 customers are on month-to-month contracts.
- 1,267 customers pay more than $70 a month and churned.
- After casting, TotalCharges has 11 nulls, which are exactly the 11 blanks identified earlier.
- Average monthly charge by contract: $66.40 (month-to-month), $65.05 (one year) and $60.77 (two year).
- Customers who pay by electronic check have a noticeably shorter average tenure (25.2 months) than those on automatic bank transfer or credit card (about 43 months).
- I saved the cleaned dataset in Parquet format. It takes up 165.3 K compared with 954.6 K for the original CSV, a reduction of about 83%, thanks to Parquet’s columnar layout and compression.
- A word count using the Spark DataFrame API returned 14,718 distinct words in 4.22 seconds.

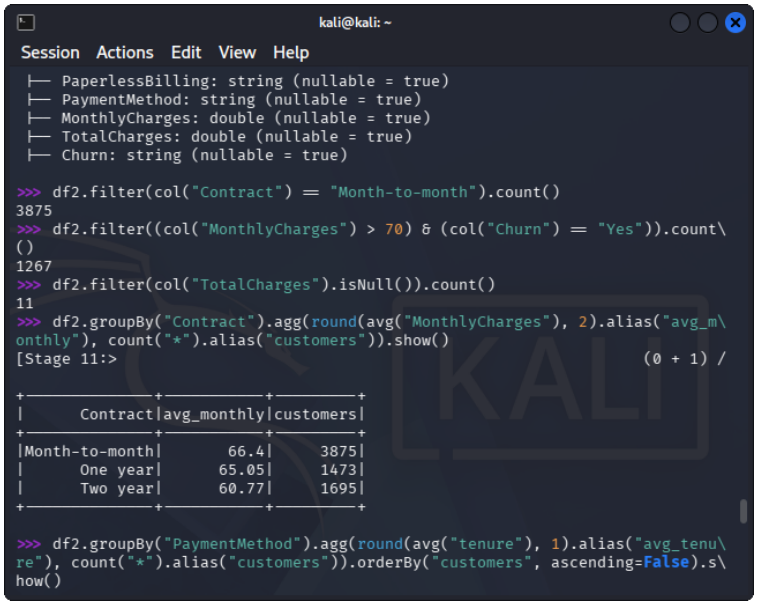


Figure 13: Filter and aggregation results: 3,875 month-to-month customers, 1,267 high-charge churners, 11 nulls after casting, and average monthly charge by contract

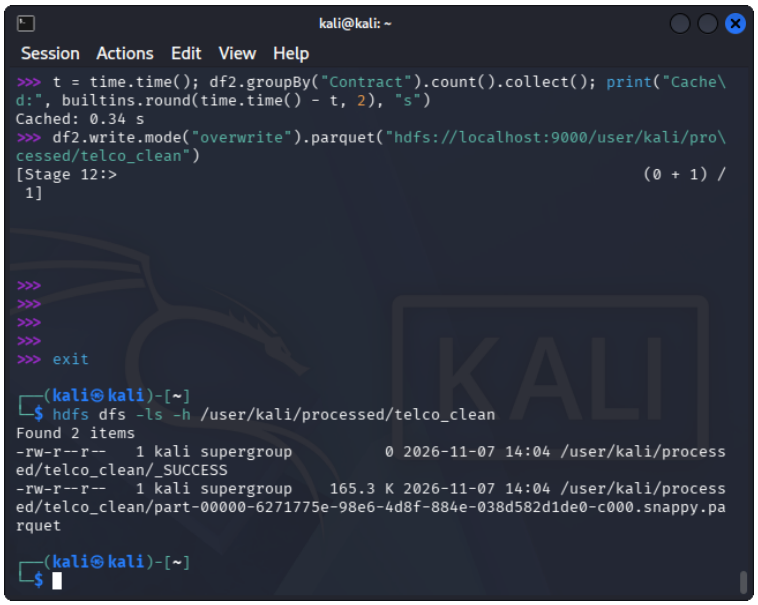


Figure 14: Writing the cleaned data to HDFS as Parquet: a single 165.3 K snappy-compressed file

Issues encountered. When I tried the same word count with the lower-level RDD API in Python, the Spark process crashed with a segmentation fault. This appears to be related to Spark's Python worker processes on my setup (see the reflection), so I switched to the DataFrame API, which runs mostly inside the JVM and completed without problems. I should also be clear that I did not run and time a Hadoop MapReduce job, so the comparison below is based on how the two engines work rather than on my own benchmark.

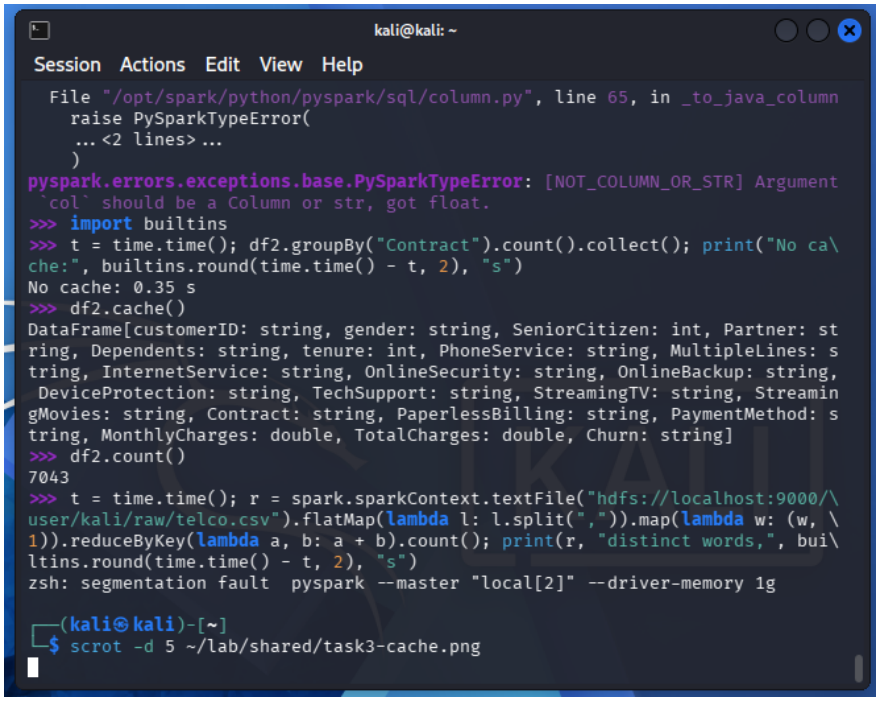


Figure 15: Timing a query before caching (0.35 s), caching the cleaned DataFrame, and the RDD word count crashing with a segmentation fault

Batch processing versus in-memory processing

	Hadoop MapReduce	Apache Spark
Processing model	Batch; intermediate results are written to disk between the map and reduce stages	In-memory; intermediate data stays in RAM across stages
Speed	Slower due to repeated disk I/O	Much faster, especially for repeated or multi-step jobs
Best suited to	Very large, one-pass batch jobs where data does not fit in memory	Iterative workloads such as machine learning and interactive queries
Resource cost	Lower RAM requirements	Higher RAM requirements

Advantages of distributed and in-memory analytics

- **Parallelism:** data is divided into partitions that are processed at the same time on different cores or nodes, so the system scales horizontally by adding machines rather than buying a bigger one.
- **Lazy evaluation:** Spark builds a logical plan (a DAG) from the transformations and only executes it when an action is called, which lets the optimiser combine steps and avoid unnecessary work.
- **Caching:** keeping a DataFrame in memory avoids re-reading it from HDFS for every query. In my own test the difference was small (0.35 s before caching and 0.34 s after), which is expected for a file under 1 MB that is read almost instantly anyway. The benefit grows as datasets get larger.
- **The trade-off is memory.** My 4 GB VM regularly hit its limit, which showed me in practice that in-memory speed is paid for in RAM. This becomes a cost decision at enterprise scale.

5. Task 4: Cost-efficient enterprise-scale architecture

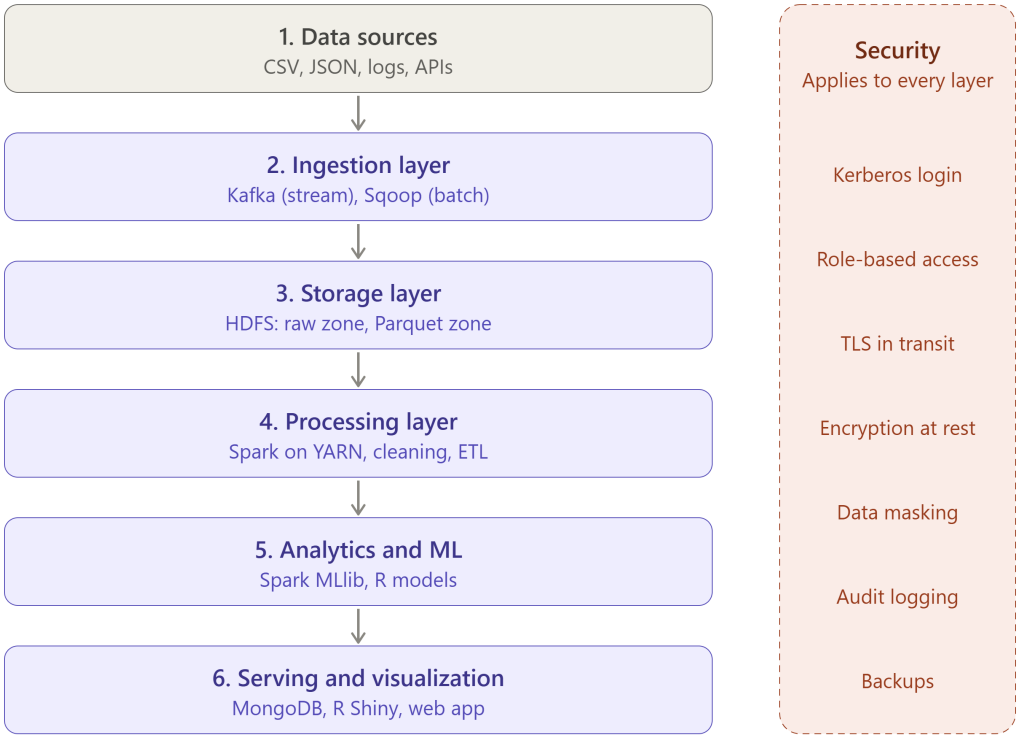


Figure 16: Proposed enterprise Big Data architecture, with security controls spanning every layer

I designed the architecture as six layers, with security treated as a cross-cutting concern rather than a separate layer, since every stage handles customer data and needs its own controls.

Layer	Technology	Role
1. Data sources	CSV, JSON, application logs, APIs	Origin of the raw data
2. Ingestion	Kafka for streaming; Sqoop or hdfs dfs -put for batch	Moves data into the platform
3. Storage	HDFS, split into a raw zone and a curated Parquet zone	Low-cost, fault-tolerant storage
4. Processing	Spark on YARN	Cleaning, transformation and ETL
5. Analytics and ML	Spark MLlib, R	Model training and scoring
6. Serving	MongoDB, R Shiny, web application	Delivers results to end users

Cost-saving strategies

- **Open-source stack:** Hadoop, Spark, Kafka, R and MongoDB Community Edition carry no licensing fees.
- **Commodity hardware and horizontal scaling:** add inexpensive nodes only when the workload requires it.
- **Efficient storage formats:** compressed columnar storage such as Parquet cut storage by about 83% in this lab.
- **Data tiering:** move older, rarely accessed data to cheaper cold storage.
- **Cloud versus on-premise:** the cloud has a low upfront cost and pay-as-you-go pricing, which suits variable workloads. On-premise needs more capital at the start but works out cheaper for heavy, steady workloads and gives the organisation direct control over sensitive data. A hybrid model can keep sensitive data on-premise while bursting compute to the cloud.

Mapping components to business use cases

Component	Business use case
Spark SQL	Report churn rates by contract and service type
Machine learning model	Identify customers at high risk of leaving
R Shiny dashboard	Let managers explore churn by customer segment
MongoDB web application	Let support staff look up an individual customer’s risk score

Justification. The design balances the three requirements in the task. It is fast because of in-memory processing and columnar storage, scalable because nodes can be added without rewriting code, and cost-efficient because it relies on open-source software and commodity hardware.

6. Task 5: Machine learning and dashboard development (R and R Shiny)

Data preparation. In R, I replaced the 11 blank TotalCharges values with 0 (based on the Task 2 finding that these are new customers), converted the categorical columns to factors, and split the data into a training set of 5,634 records (80%) and a test set of 1,409 records (20%).

Model. I trained a random forest classifier with 200 trees to predict `Churn`. A random forest handles a mix of numeric and categorical features well and reports feature importance, which is useful for explaining the results.

Evaluation on the test set

	Predicted No	Predicted Yes
Actual No	924	104
Actual Yes	190	191

Metric	Value
Accuracy	0.791
Precision	0.647
Recall	0.501
F1 score	0.565

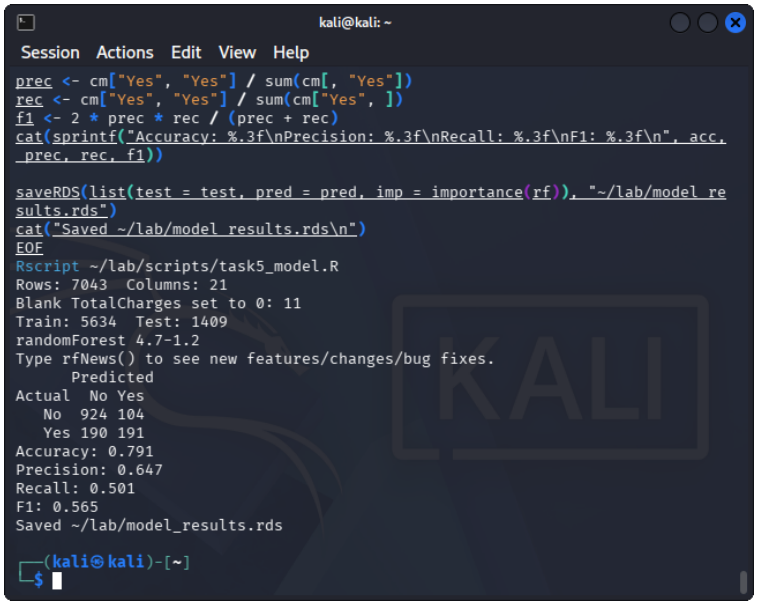


Figure 17: Running the R model script: data preparation, 80/20 split, confusion matrix and evaluation metrics

Interpreting the results. An accuracy of 79% sounds good, but a naive model that always predicted “No” would already reach about 73% because of the class imbalance. The more meaningful figure is recall, which is only 50%: the model misses about half of the customers who actually leave. For a churn use case that is a serious weakness, since those are exactly the customers the business wants to catch. Class weighting, resampling the minority class or lowering the decision threshold would be the next steps to improve recall.

Dashboard. I built an interactive R Shiny dashboard that lets the user group churn rate by contract, internet service, payment method and other attributes, filter by tenure with a range slider, and view the confusion matrix, the evaluation metrics and a feature importance chart. The top predictors were `TotalCharges`, `tenure`, `MonthlyCharges` and `Contract`. Moving the tenure slider to 12–72 months dropped overall churn from 26.5% to 17.5%, which supports the finding that newer customers are the most likely to leave.

The point of the dashboard is communication: it lets non-technical staff explore what the model found without having to read code or raw output.

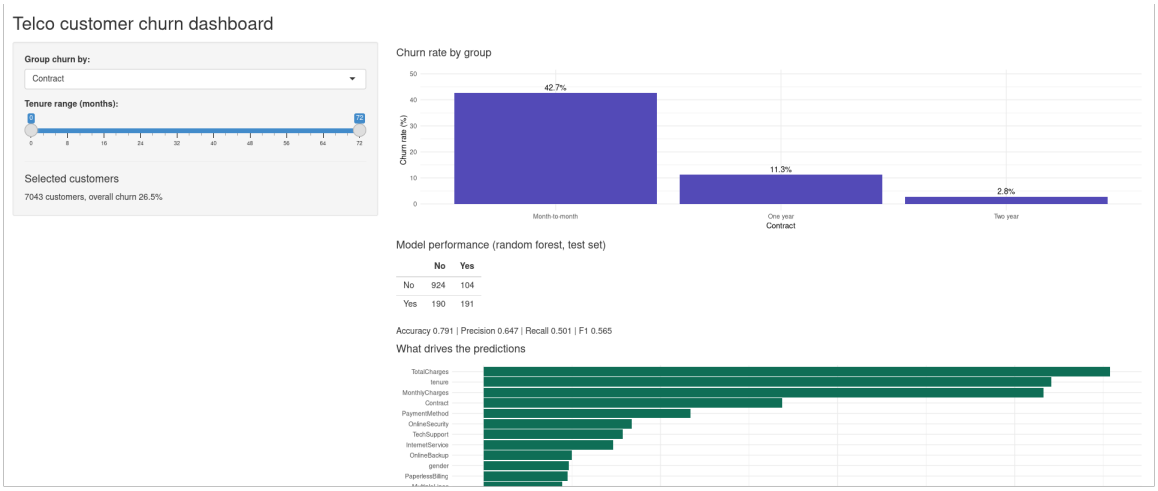


Figure 18: R Shiny dashboard grouped by contract, with the confusion matrix, metrics and feature importance

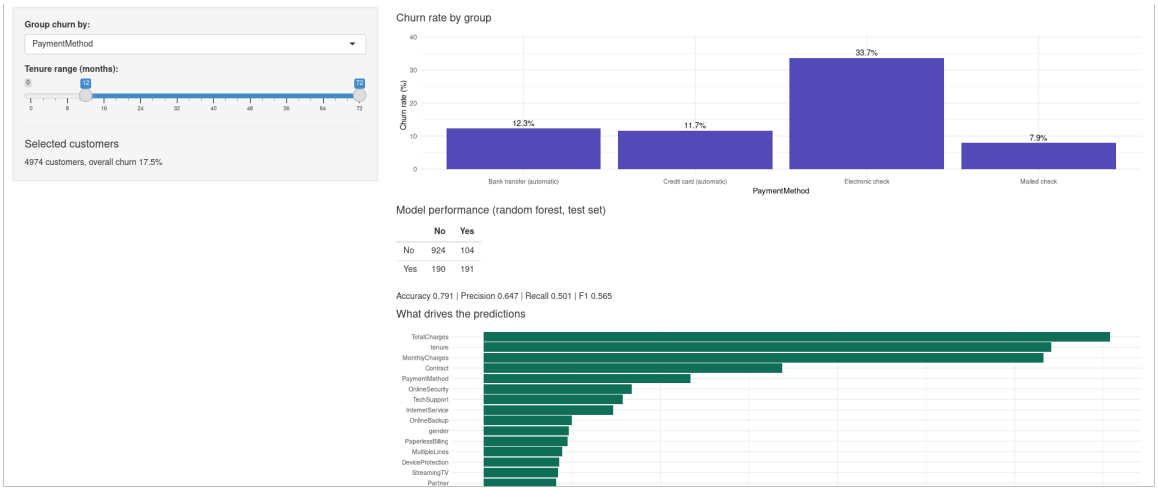


Figure 19: Tenure slider set to 12-72 months: 4,974 customers, overall churn falls to 17.5%

7. Task 6: Web-based application using a NoSQL database

Data model. I used MongoDB with a document-oriented design: one document per customer. Related attributes are embedded as sub-documents (`demographics` , `account` and `services`), alongside the `churned` flag and the model's `churn_risk` score. I also created a separate `churn_by_contract` collection holding pre-aggregated totals so that reports do not have to recompute them on every request, and added an index on `churn_risk` to speed up the highest-risk query.

MongoDB 4.4 ran in a Docker container, with port 27017 published only on 127.0.0.1 so the database was not reachable from outside the VM.

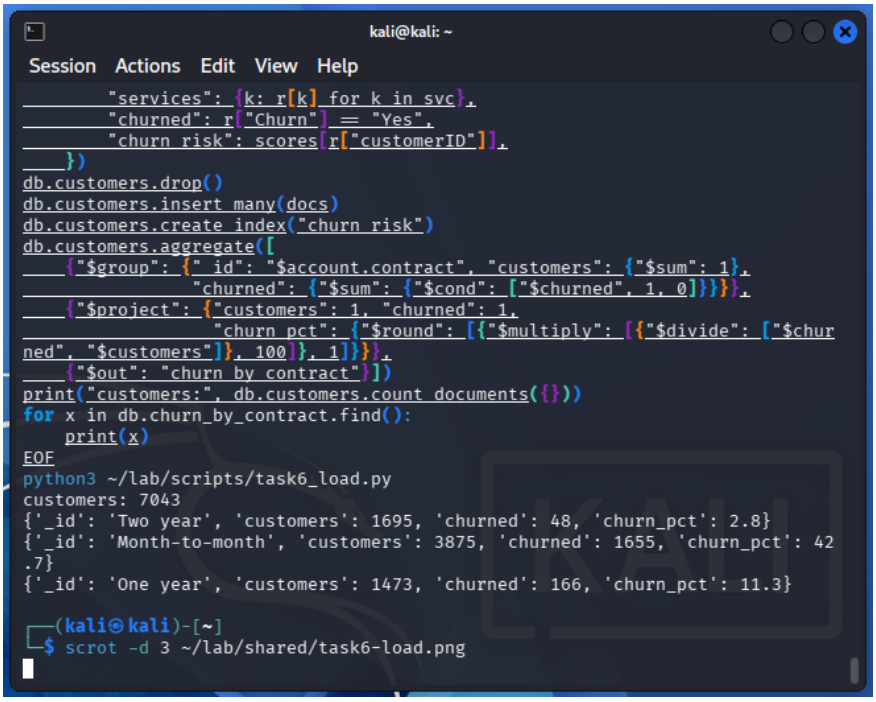
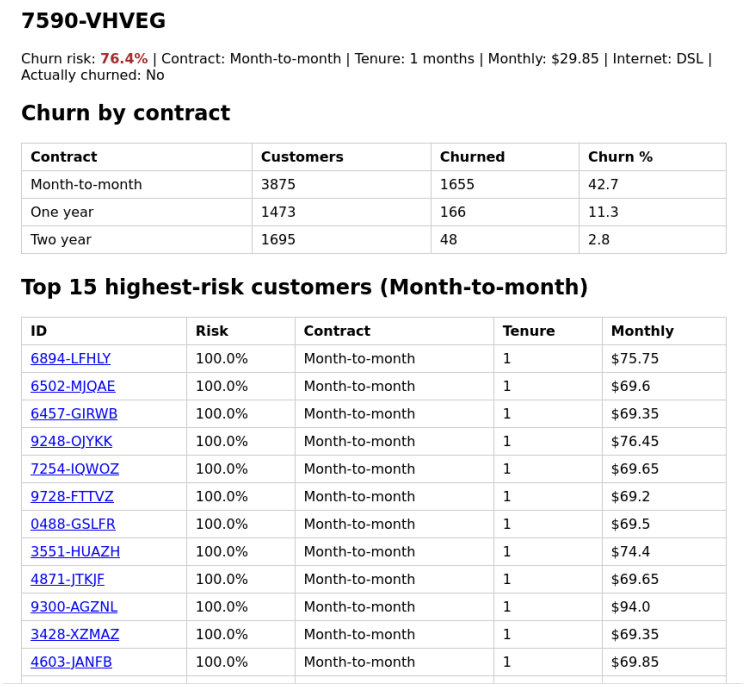


Figure 20: Loading 7,043 customer documents into MongoDB, indexing churn_risk and building the churn_by_contract collection with an aggregation pipeline

Web application. I built a lightweight Flask application that queries MongoDB. Users can look up a customer by ID, filter by contract type, and view the 15 highest-risk customers together with the churn-by-contract summary. The figures returned by MongoDB matched my Spark results exactly (3,875 / 1,473 / 1,695 customers and 42.7 / 11.3 / 2.8% churn), which gave me confidence that the data had been loaded correctly. For example, customer 7590-VHVEG is shown with a churn risk of 76.4%.



7590-VHVEG

Churn risk: **76.4%** | Contract: Month-to-month | Tenure: 1 months | Monthly: $29.85 | Internet: DSL | Actually churned: No

Churn by contract

Contract	Customers	Churned	Churn %
Month-to-month	3875	1655	42.7
One year	1473	166	11.3
Two year	1695	48	2.8

Top 15 highest-risk customers (Month-to-month)

ID	Risk	Contract	Tenure	Monthly
6894-LFHLY	100.0%	Month-to-month	1	$75.75
6502-MJQAE	100.0%	Month-to-month	1	$69.6
6457-GIRWB	100.0%	Month-to-month	1	$69.35
9248-OJYKK	100.0%	Month-to-month	1	$76.45
7254-IQWOZ	100.0%	Month-to-month	1	$69.65
9728-FTTVZ	100.0%	Month-to-month	1	$69.2
0488-GSLFR	100.0%	Month-to-month	1	$69.5
3551-HUAZH	100.0%	Month-to-month	1	$74.4
4871-JTKJF	100.0%	Month-to-month	1	$69.65
9300-AGZNL	100.0%	Month-to-month	1	$94.0
3428-XZMAZ	100.0%	Month-to-month	1	$69.35
4603-JANFB	100.0%	Month-to-month	1	$69.85

Figure 21: Flask web app: customer 7590-VHVEG with a 76.4% churn risk, the churn-by-contract table and the top 15 highest-risk month-to-month customers

Scalability and flexibility of NoSQL

- **Horizontal scalability:** MongoDB supports sharding, so data can be distributed across many servers instead of scaling up a single machine.
- **Schema flexibility:** new fields can be added to some documents without altering a fixed table schema or running a migration.
- **Efficient reads for nested data:** a single query returns the full customer record without the joins a relational design would need.
- **Trade-offs:** weaker multi-document transaction guarantees, some data duplication, and fewer built-in integrity constraints than a relational database.

8. Task 7: Information and Big Data security

Threat analysis across the data pipeline

Stage	Threat or weakness	Mitigation
Acquisition	Tampered source files; interception of data in transit	Checksum verification, TLS, schema validation on ingest
Storage (HDFS)	Overly permissive file permissions; unencrypted data at rest	POSIX permissions, ACLs, HDFS encryption zones, Kerberos
Processing (Spark)	No authentication on job submission; credentials hard-coded in scripts	Kerberos authentication, YARN queue isolation, a secrets manager
NoSQL database	Authentication disabled by default	Enable authentication and role-based access control
Visualisation and apps	No login; personal data exposed on screen	User authentication, role-based views, data masking

Access control demonstrations

- **HDFS permissions:** the raw data directory was created as `drwxr-xr-x`, meaning any user could read it. After applying `chmod 750` it became `drwxr-x---` and other users lost access. I then added an ACL entry granting a named user read access, which succeeded and confirmed that ACLs are enabled on this Hadoop version.
- **HDFS safe mode:** on start-up the NameNode stays in read-only safe mode until it has verified its block reports. My write commands were rejected during this period, which is effectively a built-in integrity control.
- **MongoDB authentication:** by default, my MongoDB instance accepted connections with no credentials at all. I enabled authentication, created an administrator account and a restricted application account with readWrite on the telco database only, and showed that anonymous access is now refused while the application account still works. This applies the principle of least privilege.
- **Network exposure:** I checked the listening ports and kept MongoDB bound to 127.0.0.1 so it is not reachable from outside the VM.

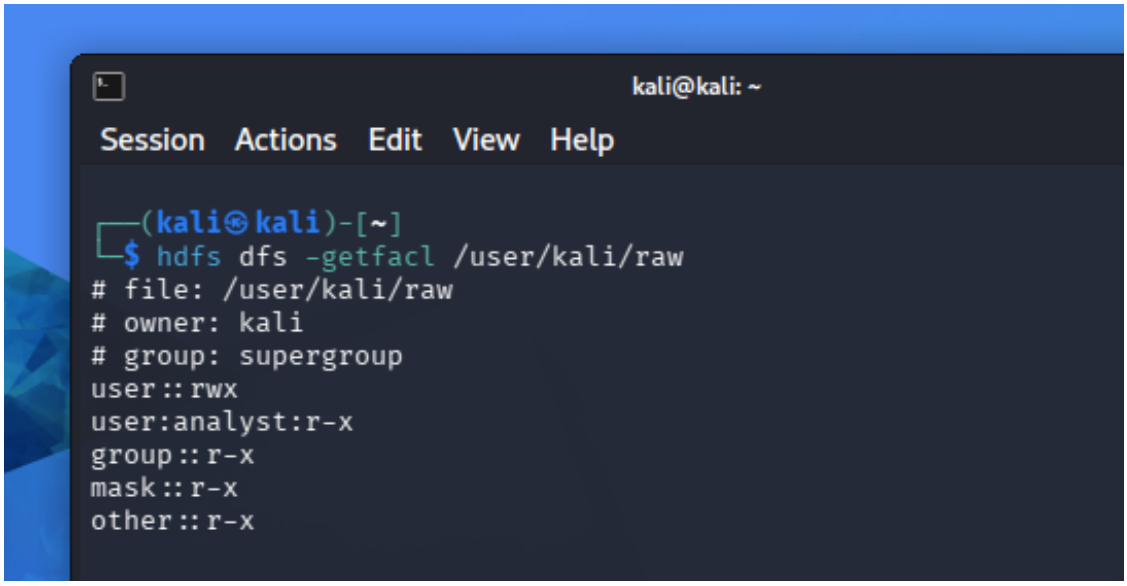


Figure 22: HDFS ACL on /user/kali/raw showing the named-user entry user:analyst:r-x



Figure 23: ss output: MongoDB listening on 127.0.0.1:27017 only, not on all interfaces

Encryption

- **Data at rest:** I encrypted the dataset with GPG using AES-256. The output is unreadable ciphertext and is actually smaller than the original (169K versus 955K) because GPG compresses the data before encrypting it. In a production cluster I would use HDFS transparent encryption zones, MongoDB's encrypted storage engine and full-disk encryption.
- **Data in transit:** TLS should be enabled on HDFS, Spark, MongoDB and the web front end. My Flask app runs over plain HTTP, which I accept as a limitation of the lab environment.
- **Key management:** during the lab I typed passwords directly into commands, which is poor practice. In production, keys and secrets belong in a dedicated key management system and never in scripts or shell history.

Compliance and privacy considerations

- Regulations such as the GDPR and the Nigeria Data Protection Act 2023 require a lawful basis for processing, data minimisation, appropriate security measures and breach notification.
- This dataset includes customer identifiers and billing information. In a real deployment I would pseudonymise customer IDs, mask sensitive fields for analysts and enforce a data retention policy.
- Big Data makes compliance harder because data is replicated across many nodes and backups, which complicates honouring a data subject's right to erasure.
- The churn risk score is itself personal data, since it profiles an individual. That means fairness and explainability of the model are compliance issues, not just technical ones.

9. Reflective summary

Working through the lab end to end helped me connect several concepts from the course to what actually happens when you build and secure a Big Data pipeline.

- **Data quality comes first.** The blank TotalCharges values passed a null check without any warning. This showed me why exploratory analysis and profiling have to come before any modelling.
- **Resources drive design decisions.** With only 4 GB of RAM I had to add swap space, run one service at a time and cap Spark's driver memory. The Spark crash during the word count made the cost and performance trade-off from the architecture task very real.
- **Version compatibility matters.** Java 25 was too new for Hadoop, so I had to install Java 21, and Python 3.14 crashed Spark's Python workers. In an organisation this is why tested, pinned software versions and proper change management are important.
- **Insecure defaults are common.** MongoDB shipped with no authentication and HDFS directories were world-readable. Security does not come automatically; it has to be configured deliberately.
- **Metrics need honest interpretation.** A 79% accuracy hides a recall of only 50%. Showing both on the dashboard means decision-makers are not misled by a single headline number.
- **Backups and recovery pay off.** When my VM got stuck in a login loop, restoring a VirtualBox snapshot recovered my work. It was a small-scale example of why backup and recovery planning matters.

Overall, the lab covered the full Big Data lifecycle, from raw files to a secured application, and showed me that architecture, cost, performance and security decisions are closely linked: a choice made in one area usually has consequences in the others.

10. Appendix: problems encountered and fixes

Problem	Fix
`openjdk-17-jdk` not available in the Kali repositories	Installed Java 21 and set it as the default JDK
Apache download site refused connections	Used a mirror site instead
Login loop in the VM	Restored a VirtualBox snapshot and avoided changing ownership of the home directory
Spark crashed on the RDD word count	Rewrote the job with the DataFrame API
Docker Hub image pull timed out	Forced IPv4 and retried the pull
Name clash between Python’s round and Spark’s `round`	Used `builtins.round` for the timing code